Capacitive Sensor Sync

Configuration and Design Reference

Politecnico di Torino

Supervisor: Prof. Mihai Teodor Lazarescu

August 13, 2026

Abstract

This document is the detailed reference for the Capacitive Sensor Sync project: a small wireless network of single-plate capacitive sensor nodes built on the STM32L412KB microcontroller and Digi XBee radios, coordinated from MATLAB. I wrote down every configuration value the system depends on, covering the clock tree, peripheral setup, pin assignment, radio parameters and packet format, so that a node can be rebuilt or a radio replaced without re-deriving anything from the source code.

The measurement method implemented in the firmware is the slope-modulation differential technique published by Subbicini, Lavagno and Lazarescu in the IEEE Sensors Journal. The repository `README.md` gives the short architectural overview; this document holds the detail.

Contents

1 System overview

The system reports the capacitance of each sensor plate on demand. A capacitive plate changes its capacitance slightly when a person moves near it, and tracking that change over time is what makes the plate useful as a presence sensor.

Four sensor nodes each measure their own plate. A MATLAB script on a PC asks all of them at once and collects the four answers, so the readings belong to the same instant rather than to four separate polls.

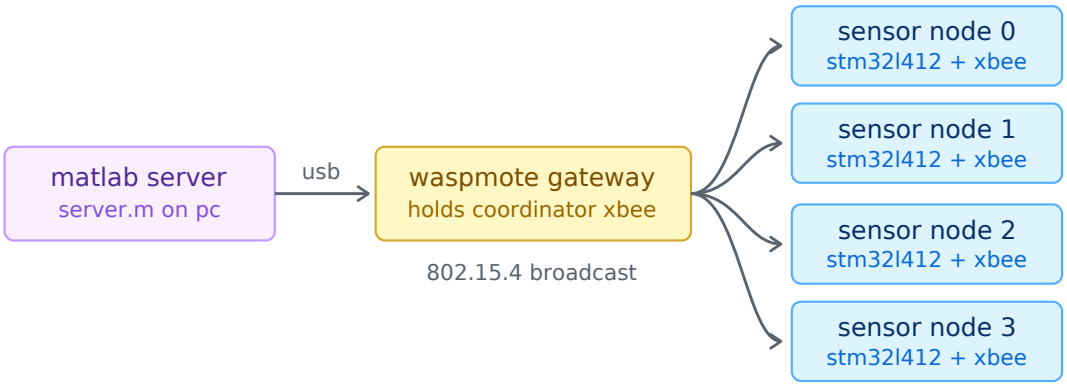

Figure 1: *System architecture. The MATLAB server drives a Waspmote gateway over USB; the coordinator radio carried by that gateway broadcasts to four identical sensor nodes.*

A full measurement round runs as follows.

1. MATLAB writes a single byte, `'r'`, to the gateway.
2. The coordinator radio broadcasts that byte. All four nodes receive it at very nearly the same moment.
3. Each node leaves its idle state and performs one capacitance measurement, taking about one millisecond.
4. Each node waits for a slot determined by its own sensor ID, then transmits a two-byte answer.
5. MATLAB collects the answers, sorts them by the ID embedded in each packet, and records anything that failed to arrive.

The staggered slots in step 4 exist because all four radios share one channel and hear the broadcast within microseconds of each other. Answering immediately, they would transmit on top of one another. Spacing the replies by sensor ID removes the contention, at the cost of a fixed 30 ms spread across a round.

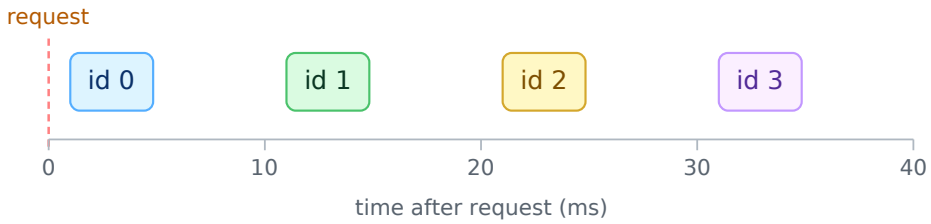

Figure 2: *Response slots. Each node delays by its sensor ID multiplied by ten milliseconds before transmitting, so the four answers arrive one after another rather than on top of each other.*

2 Hardware

Item	Detail
Microcontroller	STM32L412KBU6, Arm Cortex-M4F, UFQFPN-32 package
Node radio	Digi XBee 2.4 GHz, 802.15.4 function set
PC interface	Waspmote Gateway USB module, carrying the coordinator radio
Sensor frontend	Drift-rejection differential circuit, schematic in `Useful_Material/fediff.pdf`
Programmer	ST-Link V2 or compatible, over SWD
Firmware toolchain	Keil μVision 5, MDK-ARM
Configuration tool	STM32CubeMX v6.16.1
Radio tool	Digi XCTU
Host software	MATLAB R2019b or later

2.1 Coordinator side

The PC does not talk to a bare radio. The coordinator XBee sits in a **Waspmote Gateway** USB module, which supplies 3.3 V to the radio, provides the USB-to-serial bridge and exposes it as an ordinary COM port. MATLAB opens that port with `serialport` and writes bytes to it exactly as a sensor node writes bytes to its own radio.

The gateway doubles as the programming jig: XCTU talks to whichever module is seated in it, so I configured all five radios through it before installation, which kept the parameter set identical across the network.

2.2 Photographs

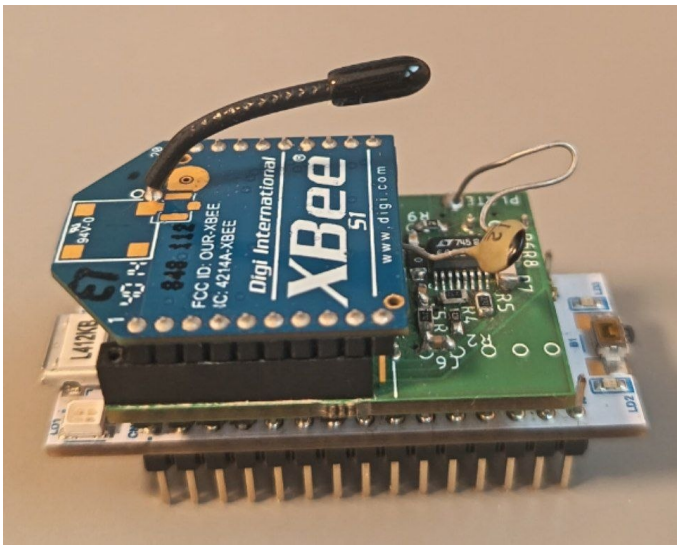

Figure 3: *Assembled sensor node: STM32L412 board, sensor frontend and XBee radio.*

Figure 4: *The same node from the side, showing the radio seated above the frontend.*

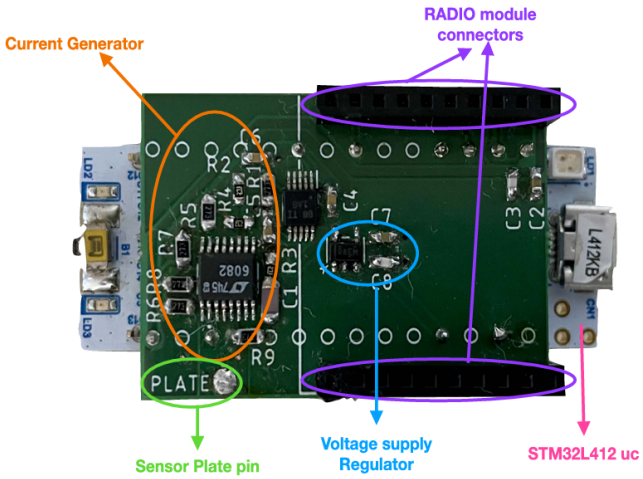

Figure 5: *Drift-rejection differential frontend.*

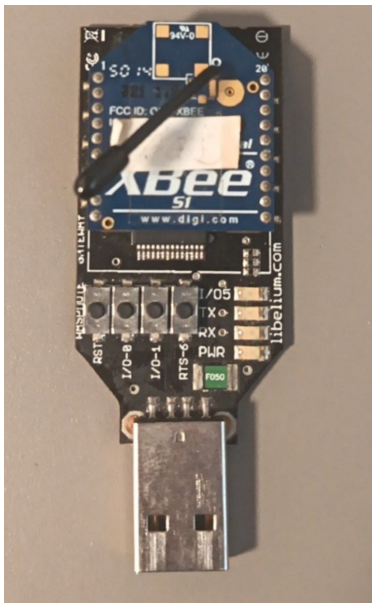

Figure 6: *Waspmote Gateway with the coordinator radio seated in it.*

Figure 7: *The XBee variants used in the project.*

3 Clock configuration

Everything in the measurement chain is derived from one system clock, which is why the two timers stay exactly in step with one another.

Parameter	Value
Oscillator	MSI, range 8, calibration value 0
PLL source	MSI
PLL dividers	M = 1, N = 10, R = /2, Q = /2
SYSCLK	80 MHz
AHB prescaler	/1
APB1 prescaler	/1
APB2 prescaler	/1
Flash latency	4 wait states
Regulator	Voltage scale 1
SysTick	1 ms, used by `HAL_Delay` and the acquisition timeout
USART1 clock	PCLK2
USART2 clock	PCLK1

Table 1: *System clock configuration, set in `SystemClock_Config()`.*

4 Pin assignment

Pin	Function	Notes
PA0	ADC1_IN5	Analogue input from the sensor frontend
PA2	USART2_TX	Debug output, AF7, debug builds only
PA9	USART1_TX	To XBee DIN, AF7
PA10	USART1_RX	From XBee DOUT, AF7
PA13	SWDIO	Programming and debug
PA14	SWCLK	Programming and debug
PA15	USART2_RX	Debug input, AF3
PB1	TIM1_CH3N	Excitation output to the frontend, AF1
PB7	GPIO output	XBee SLEEP_RQ, held low

5 Peripheral configuration

5.1 TIM1, excitation generator

TIM1 produces the square wave that drives the sensor frontend, and is also the timing master for the whole acquisition.

Parameter	Value
Prescaler	79, giving a 1 MHz counter clock from 80 MHz
Period (ARR)	999, giving a 1 kHz update rate
Counter mode	Up
Output	PWM mode 1 on channel 3, complementary output CH3N
Pulse (CCR)	500, that is 50 % duty cycle
Master output trigger	`TIM_TRGO_UPDATE`
Break and dead time	Disabled

The excitation period, 1 ms, appears in the capacitance formula as the constant `TM_SEC`.

5.2 TIM2, ADC sampling clock

TIM2 exists only to pace the ADC, and is a slave of TIM1.

Parameter	Value
Prescaler	0
Period (ARR)	39, giving a 2 MHz update rate from 80 MHz
Slave mode	`TIM_SLAVEMODE_COMBINED_RESETTRIGGER`
Input trigger	`TIM_TS_ITR0`, which is TIM1
Master output trigger	`TIM_TRGO_UPDATE`, feeding the ADC

5.3 Phase locking

Two properties together make the acquisition repeatable.

First, the ratio is exact. Both timers count the same 80 MHz clock, TIM1 is divided to 1 kHz and TIM2 to 2 MHz. That is exactly 2000 samples per excitation period, with no rounding and no slow drift between them.

Second, the phase is pinned. Because TIM2 is a slave in combined reset-trigger mode, every TIM1 update event forces the TIM2 counter back to zero. Sample number zero therefore always coincides with an excitation edge, rather than with wherever TIM2 happened to be.

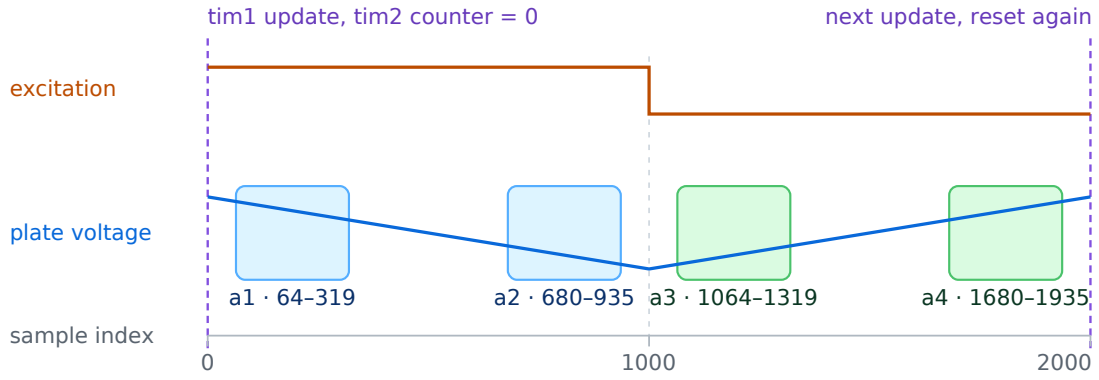

Figure 8: *One excitation period spans exactly 2000 samples. The four averaging windows are defined by sample index, and the phase lock is what keeps those indices pointing at the same part of the ramp on every acquisition.*

Without the phase lock the frame would still be 2000 samples long, but its starting offset

would differ on every acquisition, since both timers restart for each measurement. The averaging windows are defined by fixed sample indices, and those indices only carry meaning if index maps to a fixed position on the waveform.

First frame after start-up

The hardware reset only acts on a TIM1 update, so it misses the first period after the timers start. `acquire_adc_frame()` therefore clears both counters and both update flags in software first. Since the timers restart for every measurement, that first period is otherwise the first period of every measurement.

5.4 ADC1, sampling

Parameter	Value
Resolution	12 bit, right aligned
Channel	Channel 5 on PA0, single ended
Sampling time	2.5 cycles
Clock prescaler	`ADC_CLOCK_ASYNC_DIV1`
Scan mode	Disabled, one conversion per trigger
Continuous mode	Disabled
External trigger	`ADC_EXTERNALTRIG_T2_TRGO`, rising edge
DMA continuous requests	Enabled
Overrun behaviour	Data preserved
Oversampling	Disabled
Calibration	`HAL_ADCEx_Calibration_Start()` once at boot

I left continuous conversion mode off. The ADC converts once per trigger and then waits, so the sample rate is the trigger rate and nothing else needs tuning to match it. A 12-bit conversion takes 2.5 sampling cycles plus 12.5 conversion cycles, about 188 ns, comfortably inside the 500 ns between triggers.

5.5 DMA1 channel 1, transfer

Parameter	Value
Instance	`DMA1_Channel1`
Request	`DMA_REQUEST_0`, the fixed hardware mapping for ADC1
Direction	Peripheral to memory
Peripheral increment	Disabled, the data register is one fixed address
Memory increment	Enabled
Data width	Half word on both sides, matching `uint16_t`
Mode	`DMA_NORMAL`, one shot
Priority	Low
Interrupt	`DMA1_Channel1_IRQn`, priority 0

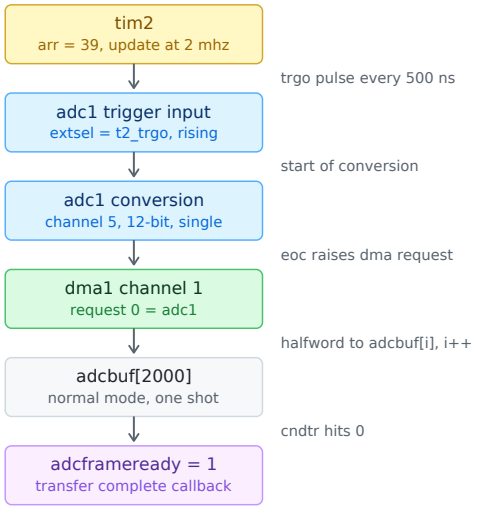

Figure 9: *From timer tick to filled buffer. Each stage is selected by a register field rather than by any physical wiring, and the whole path runs without CPU involvement until the final interrupt.*

Normal mode rather than circular is what makes this a frame instead of a stream: the channel disables itself after 2000 transfers. Circular mode would wrap and overwrite sample zero while the windows were still reading it, mixing two excitation periods into one result with nothing to show it had happened.

5.6 USART1, radio link

Parameter	Value
Baud rate	115200
Frame	8 data bits, no parity, 1 stop bit
Flow control	None
Oversampling	16
Mode	Transmit and receive, polling only
Pins	PA9 transmit, PA10 receive

Reception uses a blocking `HAL_UART_Receive()` with an infinite timeout. An interrupt handler exists in `stm32l4xx_it.c` but its NVIC line is not enabled, so the peripheral is genuinely polled.

5.7 USART2, debug port

Identical settings on PA2 and PA15. It is compiled in only when `DEBUG_UART_ENABLED` is 1, and is intended for the ST-Link virtual COM port.

5.8 GPIO and interrupts

PB7 is a push-pull output driving the XBee SLEEP_RQ line. I write the output latch low *before* switching the pin to output mode; the other order presents whatever the latch held at reset, and a high there is a valid sleep request.

The only interrupt I enabled in the NVIC is `DMA1_Channel1_IRQn`. Nothing else in the measurement path needs one, and leaving the UART interrupts masked keeps the acquisition loop uninterrupted.

6 Measurement algorithm

6.1 Frame layout

One acquisition captures `FRAME` = 2000 samples, exactly one excitation period. The first `NR` = 1000 samples cover one voltage ramp and the second 1000 cover the opposite ramp.

Four windows of `NS` = 256 samples are averaged. Each window starts `NG` = 64 samples away from a ramp edge, because the frontend is still settling immediately after a transition and those guard samples are discarded.

Window	Start index	Samples	Position
a_1	`NG`	64–319	early, first ramp
a_2	`NR - NG - NS`	680–935	late, first ramp
a_3	`NR + NG`	1064–1319	early, second ramp
a_4	`2 NR - NG - NS`	1680–1935	late, second ramp

6.2 Capacitance

Raw ADC codes are converted to volts, then the two ramp slopes are compared.

$$
\begin{aligned}
a_i &= \overline{\text{codes}_i} \cdot \frac{V_{\text{ref}}}{\text{ADC}_{\text{FS}}} \\
\Delta_{\text{fall}} &= a_2 - a_1 \\
\Delta_{\text{rise}} &= a_4 - a_3 \\
D &= \Delta_{\text{rise}} - \Delta_{\text{fall}} \\
C_p &= \frac{I \cdot T_m}{D} \cdot \frac{N_R - 2N_G - N_S}{N_R}
\end{aligned}
$$

Taking the difference of the two ramp slopes is what rejects drift. A slow, quasi-constant leakage current shifts both ramps in the same direction and cancels in D, which is the property the source paper is built around.

If $|D| < 1\,\mu\text{V}$ the result is rejected and the function returns -1, which the caller turns into the error payload. The sign of C_p depends on the physical polarity of the CH3N output, so its absolute value is taken.

6.3 Constants

Constant	Value	Meaning
SENSOR_ID	0–3	Node identity, packed into the two top bits
RESPONSE_SLOT_MS	10	Slot width, node waits ID × 10 ms
DEBUG_UART_ENABLED	0 or 1	Compiles in USART2 and text output
NR	1000	Samples per half period
FRAME	2000	Samples per acquisition
NS	256	Averaging window length
NG	64	Guard samples at each ramp edge
ADC_FS	4095	12-bit full scale
VREF	3.300 V	ADC reference voltage
I_FRONTEND	40 nA	Frontend excitation current
TM_SEC	1 ms	Excitation period
ADC_FRAME_TIMEOUT_MS	20	Guard against a frame that never completes
CAP_SCALE_PER_PF	10	Payload resolution, 0.1 pF per count
CAP_PAYLOAD_MAX	0x3FFE	Largest valid payload, 1638.2 pF
CAP_PAYLOAD_ERROR	0x3FFF	Reserved error value

Two compile-time assertions guard these values: `SENSOR_ID` fits in two bits, and `2 NG + NS` is smaller than `NR`. Both failures are silent at runtime. An oversized ID corrupts the payload bits it overflows into, and window parameters that do not fit average the wrong samples into a plausible-looking result. I made them build errors instead.

7 Communication protocol

7.1 Request

One byte. MATLAB sends lowercase `'r'` and the firmware accepts `'r'` or `'R'`. Anything else is ignored, so stray bytes on the channel do not trigger a measurement.

7.2 Response

Two bytes, most significant first.

Field	Bits	Contents
Sensor ID	15–14	Node identity, 0 to 3
Payload	13–0	Capacitance in units of 0.1 pF

Payload	Meaning
0x0000 to 0x3FFE	Valid reading, 0 to 1638.2 pF
0x3FFF	Measurement failed

Valid readings are clamped at 0x3FFE so 0x3FFF stays reserved. Every request gets an answer, including the ones where the measurement failed, so the coordinator can tell an error apart from silence. If a failure simply produced no reply, a broken frontend would look identical to a node out of radio range.

Listing 1: *Packing, from* `pack_measurement()`.

```
float scaled = cp_farad * 1.0e12f * CAP_SCALE_PER_PF;
payload = (uint16_t)(scaled + 0.5f);      /* round, then clamp to 0x3FFE */
return (uint16_t)(((SENSOR_ID & 0x0003U) << 14) | (payload & 0x3FFFU));
```

Listing 2: *Unpacking, from* `server.m`.

```
packet        = bitor(bitshift(uint16(bytes(1)), 8), uint16(bytes(2)));
transmitterID = double(bitand(bitshift(packet, -14), uint16(3)));
sensorValue   = double(bitand(packet, uint16(hex2dec("3FFF"))));
```

8 XBee radio configuration

8.1 Modules in use

Three hardware variants are present in the project.

Variant	Marking	Notes
Legacy XBee S1	`IC: 4214A-XBEE`	Standard output power
Legacy XBee-PRO S1	`IC: 4214A-XBEEPRO`	Higher transmit current
XBee S2C	`XB24CZ7PIT`	S2CTH family

The hardware revisions differ, so all three are loaded with the same Digi **XBee 802.15.4** function set in order to interoperate.

Why I reflashed the S2C modules

The S2C units arrived running Zigbee, which the legacy S1 radios cannot talk to, so I reflashed each one to the XBee 802.15.4 function set in XCTU. On Zigbee they form their own network and never appear to the coordinator, which looks like a dead node rather than a firmware mismatch.

8.2 Wiring to the STM32

XBee signal	XBee pin	STM32	Direction
VCC	1	3.3 V	—
DOUT	2	USART1_RX (PA10)	radio to MCU
DIN	3	USART1_TX (PA9)	MCU to radio
SLEEP_RQ	9	GPIO PB7	MCU to radio
GND	10	GND	—

The link settings are those of USART1 above. The rate only governs the wire between an STM32 and its own radio, not the air interface, so nodes could differ. I kept them identical anyway: a node that is out only by baud rate fails exactly like one with a radio fault.

8.3 Transparent mode

The radios run with `AP = 0`, transparent mode. Bytes written to the radio's serial input are packetised and transmitted automatically, and received payload appears directly on the receiving

radio's serial output. No API frames are constructed or parsed anywhere in the firmware, which is why the STM32 side is just `HAL_UART_Transmit` and `HAL_UART_Receive`.

8.4 Network parameters

All radios share the same function set, PAN ID (`ID`), RF channel (`CH`), MAC mode (`MM`) and encryption settings, since any mismatch silently blocks association. I picked channel `0x0C` because both the legacy XBee and the XBee-PRO support it; a channel only one of them covers would split the network along hardware lines.

Each node carries its own 16-bit source address in `MY`. For unicast back to the coordinator I set each node's `DL` to the coordinator's `MY`. The coordinator uses the broadcast destination, which is what lets one request byte reach all four nodes at once.

Device	MY	DL
Sensor node 1	1	coordinator address
Sensor node 2	2	coordinator address
Sensor node 3	3	coordinator address
Sensor node 4	4	coordinator address

Table 2: *Example addressing scheme. The radio address in `MY` is separate from the `SENSOR_ID` compiled into the firmware, and keeping the two consistent makes debugging easier.*

8.5 Sleep

Sleep is disabled in XCTU with `SM = 0`. The SLEEP_RQ pin is nevertheless wired to PB7, and the firmware drives it low at all times.

SLEEP_RQ level	Radio state
Low	Awake
High	Sleep requested

I drive the line rather than leave it floating so that enabling a pin sleep mode later cannot put a radio to sleep mid-round, which would show up as one node intermittently missing from the results.

8.6 Unused radio I/O

On the sensor PCB, XBee pins 11 and 20 are connected to ground. These are not mechanical or reserved pins, they are configurable GPIO and ADC lines: pin 11 is `AD4 / DIO4` and pin 20 is `AD0 / DIO0`.

Why pins 11 and 20 stay disabled

I configured both lines off, `D4 = 0` and `D0 = 0`, because the PCB ties them to ground. Enabled as a digital output and driven high, either would short the radio's driver straight to ground. The other unused GPIOs are off for the same reason.

8.7 XCTU settings summary

Listing 3: *Typical sensor-node profile.*

```
Function Set : XBee 802.15.4
ID           : common PAN ID
```

```
CH          : common RF channel
MY          : unique node address
DH          : 0
DL          : coordinator address
AP          : 0        # transparent mode
SM          : 0        # sleep disabled
EE          : 0        # encryption off during development
D4          : 0        # pin 11 disabled
D0          : 0        # pin 20 disabled
BD          : 115200
NB          : 0        # no parity
```

I wrote this profile and read it back in XCTU through the gateway before seating each module in its PCB. `D4` and `D0` in particular cannot be corrected safely once the module is in place.

9 Firmware behaviour

1. **Boot.** The HAL starts, the clock tree is configured, and every peripheral is initialised once. Peripherals are started and stopped for each measurement afterwards, but never re-initialised.
2. **Radio wake.** PB7 is driven low.
3. **ADC calibration.** Run once while the ADC is idle. Failure calls `Error_Handler()`.
4. **Idle.** The node blocks in `HAL_UART_Receive()` on USART1 with no timeout. A receive error aborts and re-arms the receiver rather than leaving the node deaf.
5. **Acquisition.** On `'r'` or `'R'`, both timer counters are cleared, then the ADC with DMA, TIM2 and TIM1 CH3N are started in that order. The loop waits for the transfer-complete flag with a 20 ms timeout. `stop_acquisition()` always runs afterwards, whether the frame succeeded, failed or timed out.
6. **Computation.** `compute_cp_farad()` averages the four windows and solves for capacitance.
7. **Response.** The node waits for its slot, packs the value with its ID, and transmits two bytes on USART1. In a debug build it also prints a readable line on USART2.

10 MATLAB server

10.1 Configuration

Variable	Default	Meaning
`portName`	`"COM8"`	COM port presented by the Waspmote gateway
`baudRate`	115200	Must match the radio's `BD`
`numberOfSamples`	100	Number of request rounds
`roundTimeout`	0.20 s	Longest wait for all four answers
`interRoundDelay`	0.010 s	Idle time between rounds

At start-up the script prints `serialportlist("available")`, which is the quickest way to find the right port.

10.2 Per-round behaviour

I flush the input buffer at the start of every round. A packet that arrived after the previous round timed out would otherwise be read as the first reply of the new one, shifting that node's readings a round late for the rest of the run. Packets are then read until all four IDs answer or the timeout expires. A duplicate from an ID already seen is discarded rather than overwriting the first, and nodes that never answered stay `NaN`, so a missing reading is never confused with a zero one.

10.3 Saved results

Everything is written to `four_transmitter_results.mat`.

Variable	Size	Contents
`transmitter0Values` ...	$N \times 1$	Raw payload per round, `NaN` if missing
`transmitter0Received` ...	$N \times 1$	Logical reception mask
`transmitter0ArrivalMs` ...	$N \times 1$	Arrival time in ms after the request
`sensorValues`	$N \times 4$	All four nodes, columns are IDs 0 to 3
`arrivalTimeMs`	$N \times 4$	Same layout, arrival times
`incompleteRoundCount`	scalar	Rounds with at least one node missing
`duplicatePacketCount`	scalar	Repeated answers from one ID in a round
`unexpectedPacketCount`	scalar	Stays zero, the ID field is two bits
`baudRate`, `roundTimeout`	scalar	Run configuration

Units and the error value

The stored values are raw payloads in units of 0.1 pF, and the script does not yet special-case 16383. Averaged as they stand, the sentinel enters as a 1638.3 pF reading, large enough to dominate the mean and to look like a real event. The two lines below fix both.

```
sensorValues(sensorValues == 16383) = NaN;
capacitance_pF = sensorValues / 10;
```

11 Future milestones

11.1 XBee-PRO current draw

The XBee-PRO variant draws much more current while transmitting than the standard XBee, and I have not yet checked the 3.3 V rail under transmit load on the nodes carrying one. A rail that sags during a burst also disturbs the analogue frontend, so the corrupted measurement that follows would look like a sensor fault rather than a supply problem.

11.2 Fixing the sensor boards

Several resistors and capacitors are missing from the sensor boards and need to be replaced before those nodes can be trusted. On the board I tested, R7 is gone along with the C1 connection to the inverting op-amp input at R3, and the node reports an implausibly large capacitance as a result. The other boards share the same problem to varying degrees.

11.3 Data analysis and modelling

Once all four nodes report reliable values, the next step is to remove outliers from the collected dataset and feed the four-channel data to a model of the environment at run time, which was the original purpose of the array.

Reference

G. Subbicini, L. Lavagno and M. T. Lazarescu, “Drift Rejection Differential Frontend for Single Plate Capacitive Sensors,” *IEEE Sensors Journal*, vol. 22, no. 16, pp. 16141–16149, 15 August 2022. doi:10.1109/JSEN.2022.3189031